

Afroduo Flutter Frontend

This document explains the Flutter project structure for **Afroduo**, what each folder/file contains, what it is used for, and what kind of code should go inside it.

1. Project overview

The mobile app is only the **frontend**. The backend will be in Python (for example FastAPI or Django REST Framework). Flutter will communicate with the backend through HTTP requests.

The recommended architecture is:

- **feature-first** organization: each major feature has its own folder
- **clean architecture**: presentation, domain, data layers
- **shared core**: common utilities, theme, network, constants

This makes the project scalable, readable, and easier to maintain.

2. Root folder structure

```
frontend_mobile/
├── android/
├── ios/
├── assets/
│   ├── images/
│   ├── icons/
│   ├── fonts/
│   ├── animations/
│   └── audio/
├── lib/
│   ├── main.dart
│   ├── app.dart
│   ├── core/
│   ├── shared/
│   ├── routes/
│   └── features/
├── test/
├── pubspec.yaml
└── analysis_options.yaml
```

3. Folder by folder explanation

3.1 android/

Contains everything specific to Android builds.

What goes here

- app name configuration
- Android permissions
- build.gradle settings
- icon / splash configuration for Android
- Firebase / Google services configuration if needed

What you should edit there

- internet permission if your app calls the backend
- camera, microphone, notifications permissions if needed
- Android app icon and launch settings

Example

If your app uses internet, you may need to ensure the Android manifest allows network access.

3.2 ios/

Contains everything specific to iPhone / iPad builds.

What goes here

- iOS permissions
- Xcode project settings
- app icon configuration
- iOS-specific build configuration

What you should edit there

- microphone permission if you record audio
 - internet permission if your app calls the backend
 - Apple-specific deployment settings
-

3.3 assets/

Contains static resources used by the app.

3.3.1 assets/images/

Images used in screens.

Examples: - logos - illustrations - category images - onboarding graphics - empty state images

Example file names: - `logo.png` - `lesson_banner.png` - `avatar_default.png`

3.3.2 `assets/icons/`

Custom icons used in navigation or UI.

Examples: - home icon - profile icon - audio icon - settings icon

Example file names: - `home.svg` - `mic.svg` - `progress.svg`

3.3.3 `assets/fonts/`

Custom fonts used by the app.

Examples: - `Poppins-Regular.ttf` - `Poppins-Bold.ttf` - `Inter-Medium.ttf`

Use this folder if you want a consistent visual identity.

3.3.4 `assets/animations/`

Animation files, often Lottie animations.

Examples: - loading animation - success animation - error animation - celebration animation

Example file names: - `success.json` - `loading.json`

3.3.5 `assets/audio/`

Local audio files used by the app.

Examples: - sample pronunciations - button click sounds - local tutorial audio

Example file names: - `correct.mp3` - `wrong.mp3`

4. The `lib/` folder

This is the main Dart source code folder.

Everything that defines the app logic, UI, navigation, models, services, and data flow goes here.

5. Top-level files in `lib/`

5.1 `lib/main.dart`

This is the entry point of the Flutter app.

Role

- bootstraps Flutter
- initializes services
- starts the app
- launches `App()`

What to put there

- `WidgetsFlutterBinding.ensureInitialized()`
- dependency injection setup
- initialization of shared preferences, API clients, etc.
- `runApp(const App())`

Example

```
void main() {  
  WidgetsFlutterBinding.ensureInitialized();  
  runApp(const App());  
}
```

5.2 `lib/app.dart`

This is the root widget of the application.

Role

- defines `MaterialApp`
- sets the theme
- sets the router
- configures localization if needed

What to put there

- `MaterialApp.router`
- theme settings
- route configuration
- app title

Example

```
class App extends StatelessWidget {  
  const App({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp.router(  
      title: 'Afroduo',  
      routerConfig: appRouter,  
    );  
  }  
}
```

```
        theme: AppTheme.light,  
        darkTheme: AppTheme.dark,  
    );  
}  
}
```

6. `core/` folder

This folder contains application-wide foundations shared by all features.

Nothing in `core/` should depend on a specific feature like vocabulary or quiz.

6.1 `core/config/`

Contains global configuration files.

`env.dart`

Stores environment variables or app environment constants.

Examples: - backend base URL - dev/prod mode - API keys if needed

Example:

```
class Env {  
    static const String baseUrl = 'http://10.0.2.2:8000';  
}
```

`app_config.dart`

Stores application-wide configuration.

Examples: - app name - build flavor - theme mode - default language

Use this file to centralize settings instead of hardcoding them everywhere.

6.2 `core/constants/`

Contains constants used globally.

`app_colors.dart`

Defines the app color palette.

Examples: - primary color - secondary color - background color - success/error colors

Example:

```
class AppColors {  
  static const primary = Color(0xFF2563EB);  
  static const success = Color(0xFF16A34A);  
}
```

app_strings.dart

Defines common texts.

Examples: - button labels - error messages - titles - placeholders

Example:

```
class AppStrings {  
  static const appName = 'Afroduo';  
  static const login = 'Login';  
}
```

api_constants.dart

Defines API endpoints and network-related constants.

Examples: - /auth/login - /themes - /contents

Example:

```
class ApiConstants {  
  static const login = '/auth/login';  
  static const contents = '/contents';  
}
```

6.3 core/errors/

Contains error handling classes.

exceptions.dart

Used for technical errors coming from network or data source layers.

Examples: - server exception - cache exception - unauthorized exception

`failures.dart`

Used for user-friendly failures that the UI can display.

Examples: - no internet - data not found - request failed

This separation keeps technical details away from the UI.

6.4 `core/network/`

Contains everything related to communication with the backend.

`api_client.dart`

A central HTTP client wrapper.

Use it to: - send GET/POST requests - attach headers - parse responses - handle status codes

`dio_client.dart`

Configuration for the Dio package if you use it.

Use it to define: - base URL - timeout values - request/response interceptors - authentication headers

`network_info.dart`

Checks internet connectivity.

Useful for: - showing offline messages - avoiding unnecessary requests - deciding whether to use cached data

6.5 `core/utils/`

General utility functions.

`validators.dart`

Used for form validation.

Examples: - email validation - password validation - empty input validation

`helpers.dart`

Useful generic functions.

Examples: - date formatting - string capitalization - list helpers

`formatters.dart`

Formatting helpers for values shown in the UI.

Examples: - format a date - format a number - format duration - format file size

6.6 `core/theme/`

Contains the visual design system of the app.

`app_theme.dart`

Defines the global theme.

Use it to configure: - colors - button styles - input field styles - app bar styles - light/dark theme

`text_styles.dart`

Defines reusable text styles.

Examples: - title style - subtitle style - body style - caption style

`theme_extensions.dart`

Adds custom theme values to Flutter's theme system.

Useful when the default theme is not enough.

6.7 `core/widgets/`

Contains reusable widgets used across multiple features.

`custom_button.dart`

Reusable button component.

Use it for: - login button - submit button - next button

`custom_text_field.dart`

Reusable input field.

Use it for: - email input - password input - search input

`loading_indicator.dart`

Reusable loading widget.

Use it when waiting for: - API response - audio loading - page initialization

`empty_state.dart`

Widget shown when a list is empty.

Examples: - no lessons found - no vocabulary yet - no progress data

7. `shared/` **folder**

Contains reusable structures shared between features.

7.1 `shared/models/`

Shared data models.

Examples: - API response model - pagination model - logged-in user summary

7.2 `shared/enums/`

Shared enums.

Examples: - `ContentType`: word, phrase, expression - `RequestStatus`: loading, success, error - `AppLanguage`

Example:

```
enum ContentType { word, phrase, expression }
```

7.3 `shared/providers/`

State providers used globally.

Examples: - current user provider - selected language provider - theme mode provider - authentication state provider

7.4 `shared/services/`

Business services shared by the app.

Examples: - local storage service - audio playback service - notification service - analytics service

8. routes/ folder

Contains routing and navigation logic.

8.1 app_router.dart

Defines how navigation works.

Role

- links pages to routes
- handles nested routes if needed
- manages redirects

Example use

- route to splash
 - route to login
 - route to home
 - route to lesson details
-

8.2 route_names.dart

Contains route names as constants.

Why use it

Avoids hardcoding strings everywhere.

Example:

```
class RouteNames {  
  static const splash = '/';  
  static const login = '/login';  
  static const home = '/home';  
}
```

8.3 route_guard.dart

Protects routes.

Use it when: - the user is not authenticated - a premium page requires subscription - a page requires specific permissions

Example: redirect to login if the user is not signed in.

9. features/ folder

Each feature folder contains one application module.

A feature is a complete domain area with its own UI, logic, and data handling.

10. Feature structure

A feature usually has:

```
feature_name/  
├─ presentation/  
├─ domain/  
└─ data/
```

10.1 presentation/

Contains everything visible on screen.

pages/

Full screens.

Examples: - login page - home page - vocabulary page - lesson detail page

widgets/

Small reusable UI pieces specific to that feature.

Examples: - vocabulary card - category chip - lesson tile

state/

State management for the feature.

Examples: - Bloc - Cubit - Riverpod notifier - ChangeNotifier

This folder keeps UI logic organized.

10.2 domain/

Contains business logic.

entities/

Pure objects representing core concepts.

Examples: - User - Content - Theme - Progress

repositories/

Abstract contracts describing what the data layer should provide.

Example: - VocabularyRepository - AuthRepository

usecases/

Single business actions.

Examples: - get vocabulary by theme - log in user - fetch lessons - save progress

Use cases keep business rules independent from the UI.

10.3 data/

Contains data sources and model mapping.

models/

Data models used for JSON serialization and API communication.

Example: - VocabularyModel - LessonModel - UserModel

datasources/

Where data comes from.

Examples: - backend API - local storage - cached database

repositories/

Concrete implementations of the repository contracts from domain.

Example: - VocabularyRepositoryImpl

This layer decides how to combine remote and local data.

11. Example of a feature: `vocabulary/`

```
features/vocabulary/  
├── presentation/  
│   ├── pages/  
│   ├── widgets/  
│   └── state/  
├── domain/  
│   ├── entities/  
│   ├── repositories/  
│   └── usecases/  
└── data/  
    ├── models/  
    ├── datasources/  
    └── repositories/
```

What you put inside

`presentation/pages/vocabulary_page.dart`

A screen that lists vocabulary items by theme.

`presentation/widgets/word_card.dart`

Widget showing a single word.

`domain/entities/vocabulary_item.dart`

A pure object representing a vocabulary item.

`domain/usecases/get_vocabulary_by_theme.dart`

Logic to request vocabulary items filtered by theme.

`data/models/vocabulary_model.dart`

JSON model for the API response.

`data/datasources/vocabulary_remote_datasource.dart`

Calls the backend endpoint.

`data/repositories/vocabulary_repository_impl.dart`

Converts API data into domain entities.

12. Example of a feature: `auth/`

```
features/auth/  
├── presentation/  
│   ├── pages/  
│   ├── widgets/  
│   └── state/  
├── domain/  
│   ├── entities/  
│   ├── repositories/  
│   └── usecases/  
└── data/  
    ├── models/  
    ├── datasources/  
    └── repositories/
```

Example files

- `login_page.dart`
- `register_page.dart`
- `forgot_password_page.dart`
- `auth_repository.dart`
- `login_usecase.dart`
- `auth_remote_datasource.dart`

What goes where

- UI fields and buttons go in `presentation`
- login logic goes in `domain/usecases`
- API calls go in `data/datasources`

13. Example of a feature: `splash/`

```
features/splash/  
├── presentation/  
│   ├── pages/  
│   ├── widgets/  
│   └── state/  
├── domain/  
│   ├── entities/  
│   ├── repositories/  
│   └── usecases/  
└── data/  
    ├── models/  
    ├── datasources/  
    └── repositories/
```

What splash does

- shows a logo while the app starts
- checks auth state
- loads initial data
- redirects to home or login

Example code flow

1. app launches
 2. splash page appears
 3. app checks whether the user is logged in
 4. app navigates to the proper screen
-

14. What goes in each folder in practice

Put in `pages/`

- full screens
- screens with app bars, lists, and forms

Put in `widgets/`

- reusable pieces on the same feature
- cards
- buttons
- list items

Put in `state/`

- cubits
- blocs
- notifiers
- loading / error / success state definitions

Put in `entities/`

- business objects independent of API format

Put in `usecases/`

- one action per file
- no UI code

Put in `models/`

- classes that parse JSON and map API data

Put in `datasources/`

- direct communication with backend or storage

Put in `repositories/`

- implementation that links domain and data
-

15. Example workflow for a page

Suppose you want to display vocabulary for a category.

UI flow

1. `vocabulary_page.dart` requests data from the state manager
2. state manager calls a use case
3. use case calls repository
4. repository calls remote datasource
5. datasource calls Python backend API
6. API returns JSON
7. JSON is converted to model
8. model is mapped to entity
9. page displays the result

That is the standard clean architecture flow.

16. Example of what to code where

Example: show vocabulary items

- `presentation/pages/vocabulary_page.dart` → screen and layout
 - `presentation/widgets/word_card.dart` → one word card
 - `state/vocabulary_cubit.dart` → load logic and UI state
 - `domain/entities/vocabulary_item.dart` → word object
 - `domain/usecases/get_vocabulary_by_theme.dart` → retrieval action
 - `data/datasources/vocabulary_remote_datasource.dart` → API call
 - `data/models/vocabulary_model.dart` → JSON parsing
 - `data/repositories/vocabulary_repository_impl.dart` → bridge between data and domain
-

17. Suggested starting files for Afroduo

At the beginning, create these first:

- `lib/main.dart`
- `lib/app.dart`
- `lib/core/theme/app_theme.dart`
- `lib/core/constants/app_colors.dart`
- `lib/core/network/dio_client.dart`
- `lib/routes/app_router.dart`

- `lib/features/splash/presentation/pages/splash_page.dart`
 - `lib/features/home/presentation/pages/home_page.dart`
 - `lib/features/auth/presentation/pages/login_page.dart`
 - `lib/features/vocabulary/presentation/pages/vocabulary_page.dart`
 - `lib/features/lessons/presentation/pages/lessons_page.dart`
 - `lib/features/progress/presentation/pages/progress_page.dart`
 - `lib/features/profile/presentation/pages/profile_page.dart`
-

18. Recommended conventions

File naming

Use snake_case: - `login_page.dart` - `vocabulary_repository.dart` - `custom_button.dart`

Class naming

Use PascalCase: - `LoginPage` - `VocabularyRepository` - `CustomButton`

Folder naming

Use lowercase names: - `features` - `core` - `widgets`

19. Good coding habits for this structure

- Keep UI code inside `presentation`
 - Keep API logic inside `data`
 - Keep business rules inside `domain`
 - Reuse widgets instead of duplicating them
 - Put global constants in `core/constants`
 - Keep the theme centralized in `core/theme`
 - Avoid putting everything in `main.dart`
-

20. Minimal example of a clean feature flow

Example: the login process

1. `login_page.dart` contains the form
 2. user taps the login button
 3. state manager validates inputs
 4. use case calls repository
 5. repository calls API
 6. backend responds
 7. UI receives success or error
-

21. Final summary

This structure is designed to help you build a professional Flutter app that is:

- easy to maintain
- easy to scale
- easy to understand
- compatible with a Python backend
- organized like real production projects

The most important idea is:

- **pages/widgets/state** = UI and state handling
 - **entities/usecases/repositories** = business logic
 - **models/datasources/repositories** = API and data access
 - **core** = global utilities and common code
-

22. Suggested next step

Start with only the essential folders and files, then grow the project feature by feature.

Good first feature sequence: 1. splash 2. auth 3. home 4. vocabulary 5. lessons 6. progress 7. profile 8. settings

23. Example pubspec additions

If you use the assets folders above, declare them in `pubspec.yaml`:

```
flutter:  
  assets:  
    - assets/images/  
    - assets/icons/  
    - assets/fonts/  
    - assets/animations/  
    - assets/audio/
```

24. Example package choices

Common packages for a project like this:

- `dio` for HTTP calls
- `flutter_bloc` or `flutter_riverpod` for state management
- `go_router` for navigation
- `lottie` for animations

- `shared_preferences` for local storage
 - `just_audio` or `audioplayers` for sound playback
-

If you want, I can also turn this into a **more polished README.md with badges, a table of contents, and a “project setup” section**, ready to paste directly into your Flutter repository.